JavaScript Strings

« Previous Next Chapter »

JavaScript strings are used for storing and manipulating text.

JavaScript Strings

A JavaScript string simply stores a series of characters like "John Doe".

A string can be any text inside quotes. You can use single or double quotes:

Example

```
var carname = "Volvo XC60";
var carname = 'Volvo XC60';
```

Try it Yourself »

You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

Example

```
var answer = "It's alright";
var answer = "He is called 'Johnny'";
var answer = 'He is called "Johnny"';
```

Try it Yourself »

String Length

The length of a string is found in the built in property **length**:

Example

```
var txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
var sln = txt.length;
```

Try it Yourself »

Special Characters

Because strings must be written within quotes, JavaScript will misunderstand this string:

```
var y = "We are the so-called "Vikings" from the north."
```

The string will be chopped to "We are the so-called ".

The solution to avoid this problem, is to use the **\ escape character**.

The backslash escape character turns special characters into string characters:

Example

```
var x = 'It\'s alright';
var y = "We are the so-called \"Vikings\" from the north."
```

Try it Yourself »

The escape character (\) can also be used to insert other special characters in a string.

This is the list of special characters that can be added to a text string with the backslash sign:

Code	Outputs
\'	single quote

\"	double quote
\\	backslash
\n	new line
\r	carriage return
\t	tab
\b	backspace
\f	form feed

Breaking Long Code Lines

For best readability, programmers often like to avoid code lines longer than 80 characters.

If a JavaScript statement does not fit on one line, the best place to break it is after an operator:

Example

```
document.getElementById("demo").innerHTML =
"Hello Dolly.";
```

Try it Yourself »

You can also break up a code line **within a text string** with a single backslash:

Example

```
document.getElementById("demo").innerHTML = "Hello \
Dolly!";
```

Try it Yourself »

The \ method is not a ECMAScript (JavaScript) standard.
Some browsers do not allow spaces behind the \ character.

The safest (but a little slower) way to break a long string is to use string addition:

Example

```
document.getElementById("demo").innerHTML = "Hello" +
"Dolly!";
```

Try it Yourself »

You cannot break up a code line with a backslash:

Example

```
document.getElementById("demo").innerHTML = \
"Hello Dolly!";
```

Try it Yourself »

Strings Can be Objects

Normally, JavaScript strings are primitive values, created from literals: **var firstName = "John"**

But strings can also be defined as objects with the keyword new: **var firstName = new String("John")**

Example

```
var x = "John";
var y = new String("John");

// typeof x will return string
// typeof y will return object
```

Try it Yourself »

Don't create strings as objects. It slows down execution speed.
The **new** keyword complicates the code. This can produce some unexpected results:

When using the == equality operator, equal strings looks equal:

Example

```
var x = "John";
var y = new String("John");

// (x == y) is true because x and y have equal values
```

Try it Yourself »

When using the === equality operator, equal strings are not equal, because the === operator expects equality in both type and value.

Example

```
var x = "John";
var y = new String("John");

// (x === y) is false because x and y have different types (string and
object)
```

Try it Yourself »

Or even worse. Objects cannot be compared:

Example

```
var x = new String("John");
var y = new String("John");

// (x == y) is false because x and y are different objects
// (x == x) is true because both are the same object
```

Try it Yourself »

JavaScript objects cannot be compared.

Test Yourself with Exercises!

Exercise 1 » Exercise 2 » Exercise 3 » Exercise 4 »

« Previous Next Chapter »